Three Dates, as a Container

One interface for everybody; the product that is the commitment device;
and the exact point where the four combinators stop

Claude, 21 September 2026

The brief is in `DESIGN-BRIEF.md`. In one line: a person is shown *three* candidates, must meet all three, and must choose one; matching is done by a model that interviews both sides over time; and each time they submit answers they are told how many candidates they have left, a number which is permitted to rise.

This document is the container reading. It is the sixth thing built on Neil Ghani's *Containers for Typed Agentic AI* after `tower`, `stalin`, `firm`, `federation`, `voting` and `dungeon`. The prose companion is `DESIGN.md`; this is the half with the claims in it.

1 One container

$$\mathrm{AskC} \;=\; \mathrm{Question} \triangleleft \mathrm{Reply}$$

A shape is a question; a position is a valid reply *to that question*. A free-text question admits prose, a scale admits a number in range, a choice admits one of its options, and a reply of the wrong kind is not disagreement but a type error. This is `voting`'s IssueC = Issue ◁ Vote transplanted, and the transplant is almost free.

A person is a `Runner<AskC>` — somebody a question can be put to, who answers. That is the entire ontology, and it has no notion of prolific user, lurker or newcomer in it.

Every person serves the *same* container. The family is constant, so

$$\textstyle\sum_{u\,:\,\mathrm{People}} \mathrm{AskC} \;\cong\; \mathrm{People} \times \mathrm{AskC}$$

and a sum of a constant family is a product with its index set, which does not require that index set to be finite. **The user base may be unbounded.**

This is the property `firm` had to pay for and did not get: there an agent changing the topology changed the *type*, because two sales representatives must each be told what to sell, so only a finite family could be summed. It is worth knowing what would destroy it here. Give some users a capability others lack — a verified badge with its own endpoint, a paid channel that asks directly — and the family stops being constant, taking unbounded membership with it.

1.1 Blank is a position, and the formalism insists

Principle 3 of the brief says unanswered questions are signal rather than absence. In the container reading this is not a principle anyone must remember to honour.

Claim. *A person who may decline to answer is not a container morphism unless declining is a position.*

Argument. A person who may decline is a *partial* map $(q : \text{Question}) \rightharpoonup \text{Reply}\, q$. A morphism into the do-nothing interface is a *total* dependent function $(q : \text{Question}) \to \text{Reply}\, q$; there is no arrow in **Cont** for the partial one. To have a morphism at all the positions must be widened:

$$\text{Reply}\, q \;=\; \text{ReplyFor}(\text{kind}\, q) \;+\; \text{Blank}$$

at which point declining has ceased to be a gap in the data and become an element of it — with a timestamp, a question it was declined *against*, and a place in every fold downstream. □

Blank must be in *every* fibre, for the reason above: a question that could not be declined would break the constant family and take unbounded membership with it.

2 Sign-up is the dependent pair

The first act of the brief is generative: write a question you want asked of a partner, then answer it yourself. That is $\triangleleft$ and nothing else.

```
signup.step(
  { tag: "Compose" },                  // first : "write me a question"
  (q) => ({ tag: "Ask", question: q }) // next  : that question, put back to you
)
```

The first position is the question written; `next` sends it to the shape of the author's own answer *to that question*; the composite position is the pair. A completed sign-up is therefore an element of

$$\textstyle\sum_{q\,:\,\text{Question}} \text{Reply}\, q \;=\; [\![\text{AskC}]\!]\,(1).$$

Observation. *Reciprocity is the type. "Nobody may ask what they will not answer" is not a rule enforced in front of a form; it is the statement that the sign-up act inhabits a dependent pair rather than a product. The shape cannot be built without composing a question, and the position cannot be built without answering the one that was actually composed.*

This settles one thing the brief leaves implicit. The self-answer is typed at the same Reply as everyone else's, Blank included. A person who writes a question and then declines their own has done something legible and strange, and the type records it rather than rejecting it.

3 The interviewer is the product

The brief has the system recommending questions other users submitted, and also composing new questions on demand to resolve a specific ambiguity between specific people. Both are live; exactly one question is asked.

$$\text{InterviewC} \;=\; \text{CorpusPick} \times \text{Compose}, \qquad \text{pos}\,(c, n) \;=\; \text{Reply}\, c + \text{Reply}\, n$$

Shapes multiply, positions *sum*. Prompt both sides — rank the existing corpus by information gain *and* ask the model for a question that does not exist yet — and commit to one.

> Ghani's note calls $\times$ a **prediction**: Anthropic's *Building Effective Agents* catalogues three of the four combinators, and $\times$ names a workflow the practitioner catalogue has not reached. `stalin` used it as a fallback, `federation` never used it, `firm` used it once for the CEO. Here it is the engine's inner loop *and* the commitment device of §6.

`productC` takes a `choose` policy, because a container does not choose — the program implementing $(p : P) \to R\,p$ does, and the runner is that program. That policy is the one place where the brief's distinction between an active experimenter and a passive recommender becomes a line of code, and it should be swappable, as `voting` made its cycle policy swappable, so the argument can be had against running numbers.

4 The narrowing loop, and where `seqC` lies

The loop — ask, narrow, repeat — is $\triangleleft$: the reply to one question chooses the next. But the next question is routed by kind, so the expression wanted is

$$a \triangleleft (b + c) \qquad \textit{ask, then decide}$$

and the library computes

$$(a \triangleleft b) + (a \triangleleft c) \qquad \textit{decide, then ask.}$$

A container is presented as the union of its fibres and every combinator is a conditional type, so every combinator distributes over that union. In `seqC` the second argument's shapes sit inside a *function type*, and $(P \to -)$ is a right adjoint: it preserves products and does not preserve coproducts. The rewrite has silently moved the decision to *before* the question — which is precisely the thing the brief's matching engine exists not to do, since its whole claim is that the next question is composed **after** reading the last answer.

`stalin/branch.ts` (`seqBranchC`) is the repair, and it is needed here from the first line of code. This is a soundness difference, not an ergonomic one.

In the note's own taxonomy the narrowing loop is the **evaluator–optimizer** pattern: a $\triangleleft$ chain iterated, with a $+$ gate deciding each turn whether to stop. The gate has three summands, and the third is the one the brief insists on:

$$\text{Gate} \;=\; \text{Continue} \;+\; \text{Terminate} \;+\; \text{Widen}.$$

5 The counter is a worker

The counter carries state: the corpus a person has accumulated, and their current candidate pool. Per MacBeth's third document, state needs no new machinery — a worker is a morphism whose domain has been tensored with ΔS.

With S the state set, $\Delta S := S \triangleright (\lambda_.\, S)$, and the unfolding theorem says a morphism $w : \Delta S \boxtimes \text{SubmissionC} \to \text{CountC}$ is *exactly* a pair, which then splits by the universal property of the product in the position:

$\text{fwd} : S \times \text{Submission}_0 \to \text{Count}_0$	the number the user is shown
$\text{bwd}_S : \cdots \to S$	the **writeback**: the new pool
$\text{bwd}_X : \cdots \to \text{Submission}_1$	the ordinary backward leg

Nothing was postulated. A read-state-and-answer map and a state-writeback map are consequences of the types — which is correct-by-construction for the one component of this product that everything else hangs off.

It must be the tensor, not composition. $\Delta S \boxtimes \Delta T = \Delta(S \times T)$: states multiply. $\Delta S \triangleleft \Delta T$ is store-of-store, strictly larger, and a position there carries a whole branch map $S \to T$ recording how the inner state depended on the outer. The two are an explicit retract, not an isomorphism, and the clean writeback split is specifically a tensor phenomenon. State via the tensor entangles; state via composition nests.

5.1 Principle 7, as a fact about types

The brief argues the count must be permitted to rise on grounds of honesty and resistance to gaming. The formalism makes the same point from the other end, and more cheaply.

Observation. *Nothing in a worker's type constrains the direction of the writeback.* ***The honest counter is simply the unconstrained worker.***

A progress bar is the *more* constrained object: a worker whose state space carries an order and whose writeback is required to descend it. Monotonicity is the thing one would have to go out of one's way to impose, so Principle 7 is what falls out of writing the type down at all. An implementation that finds itself adding a $\min(\text{prev}, \text{next})$ has left the algebra and should notice that it has.

6 The commitment mechanism is the product, exactly

Three candidates, meet all three, choose one:

$$\text{ThreeC} \;=\; A \times B \times C, \qquad \text{pos}\,(a, b, c) \;=\; R\,a + R\,b + R\,c.$$

Three offered, exactly one reply owed. The product is not an analogy for the mechanism; it *is* the mechanism.

Note which combinators it is not. The tensor $\boxtimes$ would owe a reply to all three — a system in which one chooses three people, a different product and a worse one. The sum $+$ would mean the choice of which candidate is under consideration was made before they were shown, which is a marketplace. Only $\times$ puts three live options in front of a person and demands a single answer.

6.1 Declining all three is a map into $(1, 0)$

There are three units, not one: $(1, 1)$ for $\boxtimes$ and $\triangleleft$, $(0, 0)$ for $+$, and $(1, 0)$ for $\times$. A morphism into $(1, 0)$ is *not* an answerer — $(1, 0)$ has no positions — it is the unique map to a terminal object. "A user may not decline all three by preference" is that sentence. Refusing the whole product is not a move in the game; it is leaving.

6.2 Therefore the carve-out is a summand

Principle 5 says declining for cause must be free and easy, and overrides everything above. There is exactly one structurally honest way to write it:

$$\text{OfferC} \;=\; (A \times B \times C) \;+\; \text{DeclineForCause}.$$

Claim. *The safety carve-out must be a summand beside the product, never a fourth position inside it.*

Argument. As a *position*, refusal-for-cause is one of the answers weighed inside the product: comparable with the other three, and therefore priceable against them. A system could then, entirely within its type, make refusal the expensive option. As a *summand* it is a different interface, reached by routing, and its cost is not comparable to anything inside the product because it is not in the product. □

> "The expense attaches to indifference, never to refusal" is the statement that the carve-out lives on the other side of a $+$. A design that puts it inside the $\times$ has violated Principle 5 in the types, before anybody writes a pricing rule.

7 Where it breaks: the assignment is not a tree

The brief decides that the three must be mutual — v appears among u's three if and only if u appears among v's — and is right that this is the hardest thing in the product. It is also the one thing the four combinators cannot express.

The combinators build **trees**: a morphism's delegate leg points down, and composites nest. A per-user narrowing is a tree and the checker is content. Mutuality is a condition on *overlaps*, and its graph has cycles. `voting` met the same wall, presenting identically:

```
const loop = seqC(you, loop);   // TS7022: referenced in its own initializer
```

This is `federation`'s subject exactly: local data — each person's three — an agreement condition on overlaps, and the question of when the local data glues to a global object. A mutual assignment *is* a global section.

7.1 What is actually hard, and what is not

The brief calls the assignment "a matching problem with quotas, closer to Gale–Shapley than to a ranking". That is the wrong reduction, and the correction changes the build order.

Claim. *A mutual three-assignment over a cohort is exactly a* 3-***factor*** *of the compatibility graph: a spanning subgraph in which every vertex has degree* 3*. By Tutte's reduction of the f-factor problem to perfect matching, its existence is decidable in polynomial time; and the maximum-weight* 3*-factor — the best three for everybody, not merely three — is computable in polynomial time by the same reduction against weighted matching.*

Mutuality is a *symmetry* constraint, not a stability one, and a degree-constrained subgraph is easier than stable matching with quotas, not harder. So the static problem is not the monster the brief fears. Three consequences are real, and the brief's architectural conclusion survives intact.

1. **It is global.** One person's answers change the feasible assignment for people they will never see. Matches cannot be computed per-user on demand, and the reason is that a 3-factor is not a local object.
2. **It is dynamic.** People arrive, answer and leave continuously. Recomputing a maximum-weight 3-factor under churn without thrashing anybody's three is the actual research problem, and the static algorithm does not touch it.
3. **Parity is a hard gate.** Summing degrees, $3n = 2|E|$, so n must be even. **Odd cohorts admit no assignment at all.**

The brief was right that this must be prototyped before any interface exists, and wrong about which part is hard. It is not the algorithm. It is the churn, and the fact that the narrowing loop must converge against a *shared* assignment rather than against one person's preferences.

7.2 A second reason the count must rise

Parity and feasibility are reasons a pool must sometimes be enlarged that have nothing to do with any individual's answers. A person will see their number go up because of somebody else's arithmetic.

This is a harder justification for Principle 7 than the psychological one, and it sharpens the brief's edge case. The system owes the user *three* sentences, not two:

I do not know enough about you	the epistemic gate
I know you, and there are not three such people here	the density problem
There are, but not in a configuration that closes	the 3-factor

8 Where the model sits, and what it costs

A language model does one thing — prompt in, answer out — so it is a direct answerer, a handler into the do-nothing interface: a **leaf**. Structure from the types, substance from the model. Three leaves here:

leaf	prompt	position
`compose`	a pool, and the ambiguity dividing it	a Question
`predict`	a person's corpus, and a question	a distribution over Reply q
`read`	two people circling one concern in different words	a judgement

`predict` is what makes information-gain ranking mean anything: a question everyone answers identically carries none and must sink, and that cannot be computed without predicting answers.

A raw model is untyped. Presenting it as $(s : S) \to P\,s$ is a *promise*, made good by a typed wrapper — a decoder at every leaf, and a reply that does not decode throws. `dungeon` took this seriously and found the caveat is not about language models at all: type discipline stops at the edge of the process, and every leaf beyond it is an oracle whether the far end is a transformer or a hand-written server.

`dungeon`'s answer is a number, and it should be copied. `rejection-rate.ts` counts every rejection against its cause and the number goes in the README. *"It sometimes fails"* is not a finding; *"it wraps the object in a markdown fence one time in twenty"* is.

Leaves are stubbed by default — deterministic, offline, fast — with real model leaves behind a flag, as `tower` and `dungeon` do it. The whole system must run and test with no credentials.

9 Which combinator is which

	in the algebra	here
$\triangleleft$	the reply to the first chooses the second	sign-up, and the narrowing loop
$\times$	both offered, exactly one answered	**the three**; and corpus-or-compose
$\boxtimes$	both shapes up front, both replies owed	state $\Delta S \boxtimes -$; and the mutual pair
$+$	routing: disjoint shapes, own positions	question kind; the gate; **the carve-out**

$\times$ doing the primary work is the notable one, and it is the first of these six repositories in which it does.

10 What the algebra refuses to decide

10.1 Is the preferred answer part of the position?

OkCupid's triple — your answer, the answer you want from a partner, an importance weight — is the prior art the brief points at, and it worked.

Against. Principle 2 says elicit acts, not claims, and "the answer I would accept" is a claim about a hypothetical person: the cheapest kind of self-report. Principle 1 says costless signals carry none, and an importance slider costs nothing.

A resolution. The question a person *chose to write* is the act-form of *what I care about*, and the set of questions they *chose to answer* is, by Principle 3, the act-form of the weight. The triple is recoverable from acts and need not be asked.

What actually changes. If the weight is asked, Reply q is a product and the corpus is dense from the first day — which is what a cold-start system wants, and cold start is the risk the brief says kills it. If it is inferred, Reply q is thin and the engine must do the work — which is exactly the work that distinguishes this from a recommender.

> The disagreement is real and should be preserved. It is the density problem arguing against the product thesis, and the right answer probably differs before and after the product has users.

10.2 The others

1. **When a cohort admits no 3-factor**, does everybody wait, or does a maximal subset get their threes while the remainder is told to keep answering? The second is kinder and quietly changes who is ever matched. `voting` met the same shape of question when a branch of a fold failed.
2. **Cadence.** How long a person may sit on three unmet candidates is a lock on three other people's assignments. In the 3-factor reading an unresolved three is a set of *pinned edges*, and pinning degrades the achievable weight for everyone else. The cost of delay is computable, which is more than the brief expected.
3. **Is information-gain ranking inside the algebra or beside it?** A container says what a valid answer is and nothing about how it was arrived at, so ranking may be entirely outside — or it is $\boxtimes$ over the population with a fold, and the corpus's self-curation is that fold's argument.
4. **What a meeting is.** Whatever else, it is a $\boxtimes$: both sides owe a reply, and a meeting with one reply is the no-show record that reputation attaches to.

11 What to build first

1. `questions.ts` — kinds, `ReplyFor<K>` and `legal`, the Blank position. Mirrors `voting`'s `issues.ts` closely to read side by side.
2. `people.ts` — `Runner<AskC>`, the corpus, and the disclosure every person owes about themselves, uniformly.
3. `assign.ts` — the 3-factor, static, on a hand-built cohort, with the parity gate and the weight. It is polynomial, it needs no model and no interface, and the three real difficulties only become visible once it runs.
4. `worker.ts` — $\Delta S \boxtimes -$, with the writeback free in both directions and a test asserting the count *rises* on a contradicting answer.
5. `branch.ts` (from `stalin`, verbatim), then `narrow.ts`.
6. `interview.ts` — the $\times$, with `choose` swappable.
7. `leaf-stub.ts`, then `leaf-claude.ts` and `rejection-rate.ts` — the number in the README.

Steps 1–4 need no network, no model and no credentials, and they contain every claim in this document that could be wrong.

12 What is assumed, and what would break it

- **The family is constant.** Every person serves AskC and nothing else. Any capability granted to some users and not others ends unbounded membership.
- **Shapes are finite unions.** `lib/algebra.ts` presents a container as the union of its fibres, which is what buys Σ over a finite index. Question *kinds* are finite; questions are not, so Question is a payload inside a fibre rather than a fibre of its own. A kind added dynamically breaks the whole conditional-type construction.
- **The oracle keeps its promise often enough.** Unmeasured, this is a hope.
- **The cohort is even, and a 3-factor exists.** Neither is generic. Both are checkable before anybody is told a number.